

ClaimMax 分层竞争（Hierarchical Competition）方案

本文描述 shared tensormap runtime 中，在现有 **ClaimMax cursor** 分片之上进一步降低同 task atomicMax 冲突的 **两轮分层竞争** 方案。目标：缩短 Claim 热路径上 atomicMax 的尾延迟，同时保持「同一 cursor 上每个 task_id 至多一个 winner」的语义。

相关现状代码：common/pa_scheduler_core.h 的 Claim()；cursor 定义见 common/pa_model.h
(alloc_cursor / cube_cursor / shared_vector_cursor 等)。

1. 背景与动机

1.1 原始单 cursor 竞争

Claim 用单调 cursor 上的 atomicMax 裁定 winner / loser：

```
old = atomicMax(&claim_max, task_id)
won = (old < task_id)
```

凡参与该 task 提交的核都要对**同一全局** claim_max 做 atomicMax 。最坏情况下全部候选核同时撞同一 cache line，atomicMax 延迟随并发竞争者近似恶化，成为 Claim 的主要开销。

1.2 现状：ClaimMax 分片

当前实现对不同 kind 使用分片 cursor，例如：

Kind	Cursor	分片数
Alloc	alloc_cursor[task_id % kCursorShards]	4
QK / PV	cube_cursor[task_id % kCursorShards]	4
SF / UP (shared)	shared_vector_cursor[task_id % kSharedVectorCursorShards]	8

分片把**不同 task_id** 的 Claim 分散到多条 AtomicLine，降低跨 task 的 false sharing，实测有收益。

但分片键是 task_id % S：**同一个 task_id 的全部候选核仍落在同一 shard**。例如 64 个 AIV 争同一个 SF/UP task 时，仍会全部

atomicMax 到同一条 shared_vector_cursor[shard] 。因此在同 task 高并发 Claim 下，仍能观察到偏长的 ClaimMax / atomicMax 时间。

1.3 下一步：分层竞争

在（可选保留的）shard cursor 之上，再按核分组做两轮竞争：

- 1. **组内轮**：先在 claim_group_max[shard][group] 上竞争；
- 2. **全局轮**：仅组内胜者再对 claim_max[shard] （即现有 shard cursor）做 atomicMax ；
- 3. **回写**：全局轮失败者把已观察到的全局水位 atomicMax 回写到本组 cursor，避免组内后续核重复冲击全局线。

常数：COMPETITION_GROUP_SIZE = 8 。

2. 方案概述

2.1 数据结构

在现有 per-shard 全局 cursor 旁增加组级 cursor （每条 AtomicLine ，与现网 Claim cursor 同宽）：

```
// 以 Vector shared 为例；Cube/Alloc 同构
// S = kSharedVectorCursorShards (或对应 kind 的分片数)
// G = ceil(num_eligible_workers / COMPETITION_GROUP_SIZE)

AtomicLine claim_max[S];           // 现有 shard cursor (全局轮)
AtomicLine claim_group_max[S][G];  // 组内轮 cursor
```

核到组的映射（固定、无动态除法即可）：

```
COMPETITION_GROUP_SIZE = 8
group_id = worker_id_in_role / COMPETITION_GROUP_SIZE
// 例：64 AIV → G=8；32 AIC → G=4；96 Alloc → G=12
```

worker_id_in_role 取该 Claim 路由下的角色内编号（AIV 用 worker_id - kAicWorkers ，AIC 用 worker_id ，Alloc 用全局 worker_id ），保证同 role 候选落在连续组内。

2.2 两轮裁定规则

轮次	操作	结果
组内	atomicMax(claim_group_max[s][g], task_id)	old_g >= task_id → loser （不再碰全局）

轮次	操作	结果
全局	组内胜者再 atomicMax(claim_max[s], task_id)	old < task_id → 唯一 winner
回写	全局失败者	atomicMax(claim_group_max[s][g], global_watermark) 后仍为 loser

不变量（与现网一致）：

- 对给定 shard cursor，推进到 task_id 时至多一个核观察到 old < task_id，该核为 winner；
- 组内失败、全局失败均为 loser；loser 不进入 Materialize / Register / Build；
- cursor 单调不减；回写只把**更高的全局水位**灌回组内，不降低任何 cursor。

2.3 为何全局失败者要回写组 cursor

组内胜者已把 claim_group_max[s][g] 至少推到 task_id，但全局可能已被其他组推到更高值 $W \geq task_id$ 。若不回写：

- 同组后续核仍可能对 (task_id, W) 再次「组内获胜」，反复冲击全局线，抵消分层收益。

因此第二轮失败者执行：

```
atomicMax(&claim_group_max[s][g], W)
```

其中 W 取全局轮返回的 old（已是写入前全局值，且 $old \geq task_id$ ），或再 load 一次全局 cursor；语义上都是把组水位抬到「至少不小于已见全局水位」。

注意：全局轮的 atomicMax(claim_max, task_id) 已经把 task_id 合并进全局 cursor。失败者**不必**再把组水位单独写回全局；需要的是 **全局 → 组** 的下行同步。

3. 伪代码

下列伪代码嵌在现有 claim() 的 kind / role 路由之后：已选好 shard 与对应的全局 cursor 指针，且本核 attempted == true。

```
CONST COMPETITION_GROUP_SIZE = 8
```

```
FUNCTION HierarchicalClaim(task_id, worker_id_in_role, shard):
    g = worker_id_in_role / COMPETITION_GROUP_SIZE
    group_cursor = &claim_group_max[shard][g]
    global_cursor = &claim_max[shard]          // 现网 shard cursor

    // ---- Round 1: 组内竞争 ----
    old_g = AtomicMax(group_cursor, task_id)
    IF old_g >= task_id:
        RETURN { attempted: true, won: false }    // 组内 loser

    // ---- Round 2: 仅组内胜者冲全局 ----
    old = AtomicMax(global_cursor, task_id)
    IF old < task_id:
        RETURN { attempted: true, won: true }     // 全局 winner

    // ---- 全局 loser: 把全局水位灌回本组 ----
    // old 已是写入前的全局值, 且 old >= task_id
    AtomicMax(group_cursor, old)
    RETURN { attempted: true, won: false }
```

与现网单轮 Claim 的对照:

```
FUNCTION FlatClaim(task_id, shard):              // 现状
    old = AtomicMax(&claim_max[shard], task_id)
    RETURN { attempted: true, won: (old < task_id) }
```

角色过滤 (非本 lane / 非候选) 仍在分层之前返回

attempted: false , 与现网一致; 分层只替换

TraceAtomicFetchMax(... ClaimMax ...) 这一段竞争。

3.1 正确性要点 (草图)

1. **全局唯一 winner**: 只有组内胜者才执行全局 atomicMax ; 全局仍是单线单调 atomicMax , 故至多一个核看到 $old < task_id$ 。
2. **组内失败 $\Rightarrow$ 全局必败**: 若组内已有 $\geq task_id$, 则该值来自本组曾成功推到至少 $task_id$ 的核; 该核会 (或已) 对全局做 $atomicMax(task_id)$, 全局终态 $\geq task_id$ 。组内后来者直接 loser 不会漏胜。
3. **回写不破坏单调性**: 只对组 cursor 做 atomicMax 抬升。
4. **与 $task_id$ 分片正交**: $shard = task_id \% S$ 仍先选线; 分层在每条 shard 线内部再减同 task 冲突。

4. 性能分析

4.1 冲突形态：分片帮不到的部分

记某次 Claim 的候选核数为 N （同 role、同 task_id）：

方案	同 task 时碰到的全局线并发上界	说明
原始单 cursor	N	全员撞一条线
仅 task_id 分片	仍为 N	同 task_id 同 shard
分层（组大小 $K=8$ ）	组内线 $\leq K$ ；全局线 $\leq \lceil N/K \rceil$	先过滤再冲全局

对 shared Vector： $N \leq 64$ ， $K = 8 \Rightarrow$ 组数 $G = 8$ ，全局轮最多约 **8** 个核同时碰 `claim_max[shard]`，相对现状最多 **64** 路碰撞，全局线竞争者上界约降为 **1/8**。

对 Cube： $N \leq 32 \Rightarrow$ 全局轮上界 **4**。
对 Alloc： $N \leq 96 \Rightarrow$ 全局轮上界 **12**。

4.2 每次 attempted Claim 的原子次数

结局	原子次数	路径
组内 loser	1	仅组内 <code>atomicMax</code>
全局 winner	2	组内 + 全局
全局 loser	3	组内 + 全局 + 组回写

稳态下多数核应落在「组内 loser」（同步浪潮中约 $(K-1)/K$ ），
平均每核原子次数 $\approx 1 + 1/K +$ (少量回写)，略高于现状的 1 次，
但：

- 组内线只在 **K=8** 核之间共享，冲突半径小；
- 全局线只承接各组胜者，冲突半径从 N 降到 $\approx N/K$ ；
- `atomicMax` 延迟通常由 **cache line 争用** 主导，而非指令条数；
用略增的原子次数换大幅下降的 per-line 争用，是本方案的收益来源。

4.3 粗力量级（同 task 全员同时 Claim）

假设争用延迟对并发度近似线性（仅作上下界直觉，非硬件模型）：

```
T_flat      ~ c * N           // 现状同 task
T_hier_grp  ~ c * K           // 组内线
T_hier_glb  ~ c * (N/K)       // 全局线（仅组胜者）
T_hier      ~ max(T_hier_grp, T_hier_glb) （两轮串行时更接近相加）
```

取 N=64 , K=8 :

```
T_flat      ~ 64c
T_hier      ~ 8c + 8c = 16c      // 组内轮 + 全局轮串行粗估
相对现状    ~ 16/64 = 0.25      // 同 task 争用路径约 4x 量级改善
```

若两轮部分重叠或回写打在已热的组线上，实际加速会落在
约 **2x-4x** 争用延迟区间，需以泳道 `claim_max child / AtomicSite::ClaimMax` 分布验证；不应把模型值直接当成无观察热路径净收益。

4.4 分层之后还要不要分片？

结论：仍然需要分片（或等价的按 **task** 分散全局线）。
分层不能替代 `task_id` 分片；二者切割的是不同冲突。

4.4.1 两种冲突正交

PA 回放里每个 worker 按 task 序 Claim，但核间进度不同：loser 路径很轻，容易跑到 winner 前面。因此硬件上会同时出现：

冲突类型	典型场景	谁缓解
同 task	64 AIV 同时 Claim 同一个 SF <code>task_id</code>	分层（组内先筛，全局最多 $\lfloor N/K \rfloor$ ）
跨 task	核 A Claim <code>t=10</code> 、核 B Claim <code>t=11</code> ，逻辑不互斥却打同一 <code>AtomicLine</code>	分片（ <code>task_id % S</code> 分到不同 line）

分片键是 `task_id % S`：同 `task_id` 必同 shard，故 **分片几乎不减同 task 的 N 路碰撞**。分层把同 task 的全局并发降到 $\approx N/K$ ，但若 `S=1`，则**所有 task** 的组胜者仍汇合到一条全局 cursor。

4.4.2 若只做分层、取消分片（S=1）

结构变为：

```
claim_max          // 唯一全局线
claim_group_max[G] // G = ceil(N/K)
```

同 task：全局并发上界 $\lfloor N/K \rfloor$ ，分层目标达成。

跨 task 与组内线却会回退：

1. 全局线重新变热

任意 shard 上的组胜者本来打 `claim_max[s]`；取消分片后，不同 `task_id` 的组胜者都打同一 `claim_max`。稳态下多核处于不同 task，全局线被「跨 task 组胜者流」持续串行化，S4.14 类分片收益被吐回。

2. 组内线也被跨 task 打满

claim_group_max[g] 若不分片，则组内 8 核的**每一个** task 的组内 atomicMax 都落在同一条组线。组线从「偶发同 task 8 路」变成「该组全流量单点」，组内轮本身可能重新偏慢。

3. 正确性仍可成立

单 cursor + 单调 task_id 的 atomicMax 在全员按序 Attempt 时语义仍对；去掉分片主要是**性能回归风险**，不是正确性必需依赖分片。分片是优化，不是 Claim 语义的前提。

粗对比 (Vector, N=64 , K=8 , G=8)：

配置	同 task 全局并发上界	跨 task 时全局线行为	组线承载
仅分片 S=8	64	分散到 8 条	无组线
仅分层 S=1	8	全部 组胜者挤 1 条	每组 1 条扛全 task 流
分层+分片 S=8	8	分散到 8 条	每组每 shard 一条，流量摊薄

可见：分层修复的是表中第一列；取消分片破坏的是第二、三列。

4.4.3 若只做分片、不做分层

即现状。跨 task 已缓解；同 task 仍最高 N 路 (Vector 64 / Alloc 96)。这正是分层要补的洞——**不能靠再加大 S** 消除：S 再大，同一 task_id 仍只对应一条 shard 线。

4.4.4 能否用「更多组 / 更深层次」代替分片？

- 把 K 减小、把 G 增大：只进一步降低**同 task** 全局并发 (上界仍是 G)，**不把不同 task_id** 分到不同全局线。
- 再加第三层 (如 socket → group → global)：同样优化的是候选核树形汇聚，全局根仍是单点，除非根也按 task_id 分叉。
- 等价替代只有一种：全局层本身按 task 分叉——那就是 **shard** (或 claim_max[task_id % S])，换名而已。

因此：**按 task 分散全局线** 与 **按核分层汇聚** 不可互相取代。

4.4.5 建议配置与可做的裁剪

选项	建议
分层 + 保留现网 S	默认： claim_max[S] + claim_group_max[S][G]
分层后增大 S	收益有限；同 task 已由分层约束，再增 S 只助跨 task，且加内存
分层后减小 / 取消 S	不建议 作默认；若做，必须 A/B 证明跨 task 全局线与组线
P99 不回退	
仅对最热 kind 分层	可先只上 shared Vector (S=8 已存在)，Cube/Alloc 随后

落地顺序建议：先 分层叠在现有分片上 量同 task ClaimMax 尾延迟；再单独做 S=8 → S=4/1 的消融。若消融显示跨 task 无回归，才考虑减片以换 GM；在消融之前不应假设「有分层即可去掉分片」。

4.5 额外成本与风险

- 1. **GM 空间**：每 shard 增加 G 条 AtomicLine (cache-line 对齐)。Vector shared： S=8 、 G=8 ⇒ +64 条；需放在 shared sidecar 并保持对齐惯例。
- 2. **胜者多 1 次原子**：唯一 winner 从 1 次变为 2 次；若胜者路径已不是瓶颈可接受，但需确认不会拖长 winner Submit。
- 3. **回写第三次原子**：仅全局失败者支付；正确性所需，需防止遗漏导致组内重复冲刷全局。
- 4. **组映射僵死**：worker_id / 8 固定分组；若某组长期空闲，只是浪费组线，不影响正确性。
- 5. **观测**：泳道若仍只打一个 ClaimMax 括号，需区分 GroupClaimMax / GlobalClaimMax / GroupSyncMax ，否则无法验证分层是否真减冲突。

4.6 预期收益小结

指标	现状（分片）	分层后预期
同 task 全局线并发上界	N	$\lceil N/K \rceil$ (K=8)
多数 loser 原子次数	1（高争用）	1（低争用组线）
Winner 原子次数	1	2
同 task atomicMax 尾延迟	仍偏高（见 loser 分析中 ClaimMax）	争用项约按 $N \rightarrow N/K$ 下降
跨 task 冲突	已由分片缓解	保持分片，不回归

结合 loser_overhead.md：空 claim.lost 的 Claim 中位约 0.7μs，其中 claim_max 中位约 0.3μs 且含插桩。分层主要切割的是高并发同 task 时 ClaimMax 的尾部，而不是把无冲突 loser 再压一个数量级。验收应看：

- 同 batch 多核同时 Claim 时 GlobalClaimMax 延迟 P99；
- 组内早退比例是否接近 (K-1)/K ；
- winner 数 / completion 与依赖签名相对基线不变。

5. 实施要点（尚未落码）

- 1. 在 pa_model.h 增加 COMPETITION_GROUP_SIZE 、 claim_group_max 布局与 AICPU reset。

2. Claim() 内用 §3 伪代码替换单次 TraceAtomicFetchMax；保留 kind/role 路由与 outcome.won = old < task_id 语义。
3. 泳道增加组/全局/回写三站点，便于和扁平 ClaimMax A/B。
4. 定向测试：同组双核 task_id 序与乱序、跨组唯一 winner、回写后组内不再误冲、与现网分片终态 cursor 一致。
5. 性能门禁：在 R5ij b256 等同配置下对比 Claim / ClaimMax 中位与 P99，并报告 winner Submit 是否回退。

6. 参考

- 现网 Claim: common/pa_scheduler_core.h → Claim()
- 分片与历史对照: shared_tensormap_record.md (S4.14 Vector cursor 等)
- Loser Submit 中 ClaimMax 占比: loser_overhead.md